Theoretical Chemistry and Computational Modelling

Programming and Computational Techniques

Python programming • Unit 0: Introduction

David Ferro Costas

Academic year 2026/2027

Why should a chemist learn to code?

Programming lets you. . .

- **automate** repetitive work: set up, run and check hundreds of calculations, not one by one
- **extract** energies, geometries or frequencies from output files, without copy–paste
- **analyse and plot** your results reproducibly
- **connect** programs: the output of one becomes the input of the next
- **understand** methods by implementing them, from a numerical integral to Hartree–Fock

Read more

Why should chemistry students learn to code? ↗

RSC, Education in Chemistry — `edu.rsc.org`

Perspectives: Teaching chemists to code ↗

ACS, Chemical & Engineering News — `cen.acs.org`

See it in action

Molecule diffusion ↗

A particle simulation in your browser

`codeskulptor.org`

Natural selection ↗

Evolution explored by simulation (video)

`youtube.com`

Why Python?

- **Readable**: a high-level language whose code looks almost like pseudocode
- **Free and open source**, on Linux, macOS and Windows
- **Versatile**: quick scripts, interactive notebooks, complete programs
- **A huge scientific ecosystem**: most computational chemistry tools can be used from Python
- **Widely used** in research and industry: plenty of documentation, examples and help online

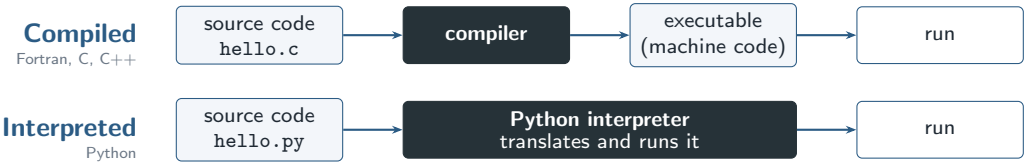
Python in (computational) chemistry

Library	Used for
NumPy, SciPy	arrays, linear algebra, numerics
Matplotlib	plots
pandas	tables of data
Jupyter	interactive notebooks
ASE	atomistic simulations
PySCF, Psi4	quantum chemistry from Python
RDKit	cheminformatics
MDAnalysis	analysis of MD trajectories
cclib	parsing outputs of QC codes

NumPy, SciPy and Matplotlib are covered in Unit 3.

How Python runs your code

Like Fortran, C or C++, Python is a **high-level** language: it is written to be read by people, and must be translated into the low-level instructions that the machine understands. The difference is *when* that translation happens.



Advantages

- No compilation step: edit and run
- Interactive use: try an idea immediately, in a terminal or a Jupyter notebook

The price: speed

- Pure Python loops are much slower than Fortran or C
- Workarounds: compiled libraries (NumPy), linking to C or Fortran (`f2py`), Cython

Strictly speaking, Python first translates your code into an intermediate *bytecode*, automatically and invisibly.

Setting yourself up

1. Install Python 3

The Anaconda distribution includes Python 3, Jupyter, NumPy, SciPy, Matplotlib and much more. You need **Python 3.10 or newer**.

2. Open a terminal

- Linux: the default terminal (bash)
- macOS: Terminal (zsh) or iTerm2
- Windows: Anaconda Prompt, Git Bash or WSL2 (Ubuntu)

3. Choose an editor

VS Code or PyCharm (both have free versions), or Vim and Emacs if you already use them.

Three ways to run Python

```
$ python3 my_program.py  # a script
$ ipython                # interactive
$ jupyter lab            # notebooks
```

Check the version:

```
$ python3 --version
Python 3.x.y
```

`python` or `python3`? These slides write `python3`; on Windows (Anaconda Prompt) type `python`. Not found, or Python 2? Then Anaconda is not active in that terminal.

No installation? CodeSkulptor3 runs Python 3 in your browser.

Your first Python program

In a text editor, write these lines and save them as `hello_world.py`:

hello_world.py

```
#!/usr/bin/env python3
# Our first Python program
print("Hello World!")
```

- Everything after `#` is a **comment**: ignored by Python, written for people
- `print()` writes text on the screen
- The first line ("shebang") tells Linux and macOS which interpreter runs the file

Windows. Run `python hello_world.py`; the first line is ignored.

Run it from a terminal

```
$ python3 hello_world.py
Hello World!
```

or make it executable (Linux, macOS):

```
$ chmod +x hello_world.py
$ ./hello_world.py
Hello World!
```

Interactive mode: type `python3` alone

```
$ python3
>>> print("Hello World!")
Hello World!
>>> exit()
```

How to read these slides

- **Scripts** (code saved in a `.py` file) have a blue bar; what they print appears in a dark box
- **Interactive sessions** have a yellow bar and the `>>>` prompt; results are shown in orange
- Interactive mode (and Jupyter) displays the value of an expression automatically; in a script you need `print()`
- Slides tagged EXTRA go beyond the essentials

Script and its output

```
energy = -76.0268      # hartree
print("E =", energy)
```

```
E = -76.0268
```

Interactive session

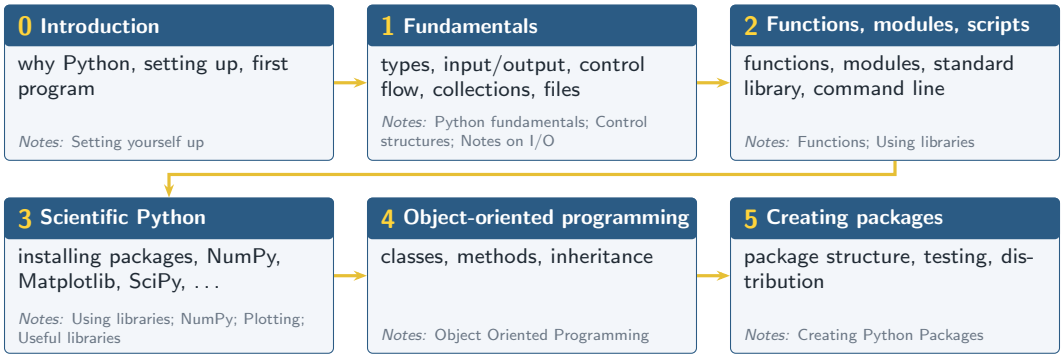
```
>>> 7 / 2
3.5
```

Python 3. All the code in this course is Python 3 ($\geq$ 3.10). Python 2 is obsolete since 2020: avoid old code written for it.

Course roadmap

lecture notes ↗

Six units. Each one builds on the previous ones and follows the lecture notes (link above):



Resources

Online

CodeSkulptor3 ↗

Python 3 in the browser, and its documentation

`py3.codeskulptor.org • docs`

Python documentation ↗

The official reference and tutorial

`docs.python.org/3`

Python cheat sheet ↗

The essentials of the language on one page

`www.pythoncheatsheet.org`

Think Python ↗

A. Downey; Books and Notebooks via Colab

`https://allendowney.github.io/ThinkPython/`

Books

- *A Primer on Scientific Programming with Python*
  H. P. Langtangen **PDF ↗**
- *Learning Python*
  M. Lutz **PDF ↗**
- *Programming for Computations – Python*
  S. Linge, H. P. Langtangen **PDF ↗**
- *Python for Everybody*
  C. Severance **PDF ↗**

Beginner's Python Cheat Sheet

Variables and Strings

Variables are used to store values. A string is a series of characters, surrounded by single or double quotes.

Hello world

```
print("Hello world!")
```

Hello world with a variable

```
msg = "Hello world!"
print(msg)
```

Concatenation (combining strings)

```
first_name = 'albert'
last_name = 'einstein'
full_name = first_name + ' ' + last_name
print(full_name)
```

Lists

A list stores a series of items in a particular order. You access items using an index, or within a loop.

Make a list

```
bikes = ['trek', 'redline', 'giant']
```

Get the first item in a list

```
first_bike = bikes[0]
```

Get the last item in a list

```
last_bike = bikes[-1]
```

Looping through a list

```
for bike in bikes:
    print(bike)
```

Adding items to a list

```
bikes = []
bikes.append('trek')
bikes.append('redline')
bikes.append('giant')
```

Making numerical lists

```
squares = []
for x in range(1, 11):
    squares.append(x**2)
```

Lists (cont.)

List comprehensions

```
squares = [x**2 for x in range(1, 11)]
```

Slicing a list

```
finishers = ['sam', 'bob', 'ada', 'bea']
first_two = finishers[:2]
```

Copying a list

```
copy_of_bikes = bikes[:]
```

Tuples

Tuples are similar to lists, but the items in a tuple can't be modified.

Making a tuple

```
dimensions = (1920, 1080)
```

If statements

If statements are used to test for particular conditions and respond appropriately.

Conditional tests

```
equals            x == 42
not equal         x != 42
greater than      x > 42
  or equal to     x >= 42
less than         x < 42
  or equal to     x <= 42
```

Conditional test with lists

```
'trek' in bikes
'surly' not in bikes
```

Assigning boolean values

```
game_active = True
can_edit = False
```

A simple if test

```
if age >= 18:
    print("You can vote!")
```

If-elif-else statements

```
if age < 4:
    ticket_price = 0
elif age < 18:
    ticket_price = 10
else:
    ticket_price = 15
```

Dictionaries

Dictionaries store connections between pieces of information. Each item in a dictionary is a key-value pair.

A simple dictionary

```
alien = {'color': 'green', 'points': 5}
```

Accessing a value

```
print("The alien's color is " + alien['color'])
```

Adding a new key-value pair

```
alien['x_position'] = 0
```

Looping through all key-value pairs

```
fav_numbers = {'eric': 17, 'ever': 4}
for name, number in fav_numbers.items():
    print(name + ' loves ' + str(number))
```

Looping through all keys

```
fav_numbers = {'eric': 17, 'ever': 4}
for name in fav_numbers.keys():
    print(name + ' loves a number')
```

Looping through all the values

```
fav_numbers = {'eric': 17, 'ever': 4}
for number in fav_numbers.values():
    print(str(number) + ' is a favorite')
```

User input

Your programs can prompt the user for input. All input is stored as a string.

Prompting for a value

```
name = input("What's your name? ")
print("Hello, " + name + "!")
```

Prompting for numerical input

```
age = input("How old are you? ")
age = int(age)

pi = input("What's the value of pi? ")
pi = float(pi)
```


Python Crash Course

Covers Python 3 and Python 2

nostarchpress.com/pythoncrashcourse



Python For Data Science *Cheat Sheet*

Python Basics

Learn More Python for Data Science Interactively at www.datacamp.com



Variables and Data Types

Variable Assignment

```
>>> x=5
>>> x
5
```

Calculations With Variables

```
>>> x+2
7
>>> x-2
3
>>> x*2
10
>>> x**2
25
>>> x%2
1
>>> x/float(2)
2.5
```

Sum of two variables

Subtraction of two variables

Multiplication of two variables

Exponentiation of a variable

Remainder of a variable

Division of a variable

Types and Type Conversion

`str()`	`'5', '3.45', 'True'`	Variables to strings
`int()`	`5, 3, 1`	Variables to integers
`float()`	`5.0, 1.0`	Variables to floats
`bool()`	`True, True, True`	Variables to booleans

Asking For Help

```
>>> help(str)
```

Lists

Also see NumPy Arrays

```
>>> a = 'is'
>>> b = 'nice'
>>> my_list = ['my', 'list', a, b]
>>> my_list2 = [[4,5,6,7], [3,4,5,6]]
```

Selecting List Elements

Index starts at 0

Subset

```
>>> my_list[1]
>>> my_list[-3]
```

Select item at index 1

Select 3rd last item

Slice

```
>>> my_list[1:3]
>>> my_list[1:]
>>> my_list[:3]
>>> my_list[:]
```

Select items at index 1 and 2

Select items after index 0

Select items before index 3

Copy my_list

Subset Lists of Lists

```
>>> my_list2[1][0]
>>> my_list2[1][:2]
```

my_list[list][itemOfList]

List Operations

```
>>> my_list + my_list
['my', 'list', 'is', 'nice', 'my', 'list', 'is', 'nice']
>>> my_list * 2
['my', 'list', 'is', 'nice', 'my', 'list', 'is', 'nice']
>>> my_list2 > 4
True
```

List Methods

```
>>> my_list.index(a)
>>> my_list.count(a)
>>> my_list.append('!')
>>> my_list.remove('!')
>>> del(my_list[0:1])
>>> my_list.reverse()
>>> my_list.extend('!')
>>> my_list.pop(-1)
>>> my_list.insert(0,'!')
>>> my_list.sort()
```

Get the index of an item

Count an item

Append an item at a time

Remove an item

Remove an item

Reverse the list

Append an item

Remove an item

Insert an item

Sort the list

Strings

```
>>> my_string = 'thisStringIsAwesome'
>>> my_string
'thisStringIsAwesome'
```

String Operations

```
>>> my_string * 2
'thisStringIsAwesomethisStringIsAwesome'
>>> my_string + 'Innit'
'thisStringIsAwesomeInnit'
>>> 'm' in my_string
True
```

String Operations

Index starts at 0

```
>>> my_string[3]
>>> my_string[4:9]
```

String Methods

```
>>> my_string.upper()
>>> my_string.lower()
>>> my_string.count('w')
>>> my_string.replace('e', 'i')
>>> my_string.strip()
```

String to uppercase

String to lowercase

Count String elements

Replace String elements

Strip whitespaces

Libraries

Import libraries

```
>>> import numpy
>>> import pandas as pd
```

Selective import

```
>>> from math import pi
```

pandas — Data analysis

scikit-learn — Machine learning

NumPy — Scientific computing

matplotlib — 2D plotting

Install Python



Leading open data science platform powered by Python



Free IDE that is included with Anaconda



Create and share documents with live code, visualizations, text, ...

Numpy Arrays

Also see Lists

```
>>> my_list = [1, 2, 3, 4]
>>> my_array = np.array(my_list)
>>> my_2darray = np.array([[1,2,3],[4,5,6]])
```

Selecting Numpy Array Elements

Index starts at 0

Subset

```
>>> my_array[1]
2
```

Select item at index 1

Slice

```
>>> my_array[0:2]
array([1, 2])
```

Select items at index 0 and 1

Subset 2D Numpy arrays

```
>>> my_2darray[:,0]
array([1, 4])
```

my_2darray[rows, columns]

Numpy Array Operations

```
>>> my_array > 3
array([False, False, False,  True], dtype=bool)
>>> my_array * 2
array([2, 4, 6, 8])
>>> my_array + np.array([5, 6, 7, 8])
array([6, 8, 10, 12])
```

Numpy Array Functions

`>>> my_array.shape`	Get the dimensions of the array
`>>> np.append(other_array)`	Append items to an array
`>>> np.insert(my_array, 1, 5)`	Insert items in an array
`>>> np.delete(my_array,[1])`	Delete items in an array
`>>> np.mean(my_array)`	Mean of the array
`>>> np.median(my_array)`	Median of the array
`>>> my_array.corrcoef()`	Correlation coefficient
`>>> np.std(my_array)`	Standard deviation